

Ultimate Git & GitHub Cheat Sheet

QUICK REFERENCE GUIDE FOR VS CODE & MACOS CONSOLE

This master cheat sheet contains all the workflows and terminal commands to create repositories, scrub broken histories, safely exclude environments/configurations, and maintain updates daily.

1. Managing Repositories on GitHub

Option A: Creating a New Clean Repository

1. Log into **GitHub.com**.
2. Click the "+" icon top-right → **New repository**.
3. Set a name (e.g., `learning-hub`) and choose public/private.
4. **CRITICAL**: Leave "Add a README file", ".gitignore template", and "License" **UNCHECKED**.
5. Click **Create repository** and copy the HTTPS URL string.

Option B: Cloning (Downloading) an Existing Repo to Your Mac

```
# Navigate to your workspace directory
cd /path/to/projects_folder

# Download the project from GitHub
git clone https://github.com/your-username/your-repo-name.git

# Move inside the downloaded project directory
cd your-repo-name
```

2. Wiping History & Starting Fresh (The Clean Slate)

Use this if you accidentally committed massive virtual environments or `.env` files earlier and need to clear them entirely from tracking history:

```
# Wipe out the local tracking history database entirely
rm -rf .git

# Initialize a completely fresh local repository
git init
```

3. Ignoring Sensitive Configurations & Virtual Envs

Create strict exclusion parameters inside your local root environment:

```
# Append exclusion properties to your .gitignore
echo ".env" >> .gitignore
echo "venv/" >> .gitignore
echo ".venv/" >> .gitignore
echo "env/" >> .gitignore
echo "__pycache__/" >> .gitignore
echo "*.pyc" >> .gitignore
```

Post-Clone Recovery Tip: When cloning down old code via Option B, your `.env` and environment environments are excluded dynamically. Reconstruct them using:

```
python3 -m venv venv && source venv/bin/activate
```

4. Auditing, Committing & Initial Push

```
# 1. Stage all files (Git ignores parameters set in Step 3)
git add .

# 2. AUDIT THE PAYLOAD - Verify nothing green shouldn't be there
git status

# 3. Create your official baseline local commit
git commit -m "Initial clean setup: ignoring configurations and virtual environments"

# 4. Standardize local primary branch context alignment
git branch -M main

# 5. Connect local infrastructure to your GitHub project URL
git remote add origin https://github.com/your-username/your-repo-name.git

# 6. Force-push to overwrite any messy remote legacy files on GitHub
git push -u origin main --force
```

 **Security:** Force-pushing overrides UI timelines on GitHub, but individual old commit objects can remain cached. If you exposed live cloud tokens or keys, **rotate them immediately**.

5. Everyday Production Cycle Cheat Sheet

Saving Ongoing Changes

```
# Check current drift status
git status

# Stage specific modified folder target
git add folder_name/

# Or stage every local adjustments
git add .

# Commit changes with explicit documentation message
git commit -m "Refactor processing loop configuration parameters"
```

Syncing with GitHub Workspace

```
# Push local commits upwards to GitHub
git push

# Pull down recent modifications completed directly on GitHub browser interface
git pull
```

Isolating Features Safely (Branching)

```
# Spin up and jump into a brand new standalone dev branch
git checkout -b feature/analytics-core

# Jump back to production main line code branch
git checkout main

# Merge finalized functional feature logic context back into main branch
git merge feature/analytics-core
```